



Informe Tarea 3 Sistema Centro de Apoyo Académico

NRC 8025 – Viña del Mar

Opción B – Implementación en Java (Stream API)

Alumno: Felipe Briceño – 21.340.567-5

Asignatura: PTEC102 – Paradigmas de Programación

Opción elegida: B – Java funcional con Stream API

1. Descripción del sistema

El sistema implementado corresponde al Centro de Apoyo Académico, desarrollado en Java utilizando la API Stream y expresiones lambda. El programa mantiene una base de conocimiento con 30 estudiantes, cada uno con nombre, curso y nota, y permite realizar consultas interactivas por consola mediante un menú con bucle do-while.

2. Estructura del programa

2.1 Clase Estudiante

Clase auxiliar que encapsula los atributos de un estudiante: nombre, curso y nota. Incluye constructor y getters.

2.2 Clase CentroApoyo (principal)

Método	Descripción	Usa Stream
cargarEstudiantes()	Inicializa la lista con 30 objetos Estudiante	No
perteneceACurso(nombre, curso)	Verifica pertenencia con filter + anyMatch	Sí
estaAprobado(nombre)	Verifica nota ≥ 4.0 con filter + anyMatch	Sí
estaReprobado(nombre)	Verifica nota < 4.0 con filter + anyMatch	Sí
mostrarEstadoEstudiante()	Lee nombre con Scanner, busca con findFirst	Sí
calculadora()	Solicita dos números y una operación básica	No
menu()	Menú interactivo con do-while hasta que el usuario salga	No

3. Código fuente comentado

A continuación, se presenta el código completo del archivo CentroApoyo.java, con comentarios Javadoc en cada método (comentarios de color azul para facilitar la lectura de comentarios).

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

/**
 * Representa a un estudiante con su nombre, curso y nota.
 */
class Estudiante {
    private String nombre;
    private String curso;
    private double nota;

    public Estudiante(String nombre, String curso, double nota) {
        this.nombre = nombre;
        this.curso = curso;
        this.nota = nota;
    }

    public String getNombre() { return nombre; }
    public String getCurso() { return curso; }
    public double getNota() { return nota; }
}

/**
 * Sistema Centro de Apoyo Academico.
 * Permite consultar estudiantes, verificar pertenencia a cursos y usar una calculadora.
 */
public class CentroApoyo {

    static List<Estudiante> estudiantes = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        cargarEstudiantes();
        menu();
    }

    static void cargarEstudiantes() {
        estudiantes.add(new Estudiante("ana", "prolog", 6.5));
        estudiantes.add(new Estudiante("luis", "prolog", 3.8));
        estudiantes.add(new Estudiante("maria", "prolog", 5.2));
        estudiantes.add(new Estudiante("pedro", "prolog", 4.0));
        estudiantes.add(new Estudiante("sofia", "prolog", 2.9));
        estudiantes.add(new Estudiante("carlos", "java", 5.8));
        estudiantes.add(new Estudiante("valentina", "java", 4.5));
        estudiantes.add(new Estudiante("diego", "java", 3.5));
        estudiantes.add(new Estudiante("camila", "java", 6.0));
        estudiantes.add(new Estudiante("tomas", "java", 1.0));
        estudiantes.add(new Estudiante("fernanda", "python", 5.5));
    }
}
```

```

    estudiantes.add(new Estudiante("nicolas", "python", 3.9));
    estudiantes.add(new Estudiante("isadora", "python", 6.8));
    estudiantes.add(new Estudiante("matias", "python", 4.2));
    estudiantes.add(new Estudiante("renata", "python", 2.5));
    estudiantes.add(new Estudiante("ignacio", "redes", 4.7));
    estudiantes.add(new Estudiante("javier", "redes", 3.0));
    estudiantes.add(new Estudiante("rodrigo", "redes", 5.0));
    estudiantes.add(new Estudiante("constanza", "redes", 3.2));
    estudiantes.add(new Estudiante("andres", "redes", 6.1));
    estudiantes.add(new Estudiante("daniela", "matematica", 4.8));
    estudiantes.add(new Estudiante("felipe", "matematica", 3.6));
    estudiantes.add(new Estudiante("catalina", "matematica", 5.3));
    estudiantes.add(new Estudiante("esteban", "matematica", 2.0));
    estudiantes.add(new Estudiante("paola", "matematica", 6.2));
    estudiantes.add(new Estudiante("gonzalo", "prolog", 4.1));
    estudiantes.add(new Estudiante("barbara", "java", 3.7));
    estudiantes.add(new Estudiante("hector", "python", 5.9));
    estudiantes.add(new Estudiante("lorena", "redes", 4.3));
    estudiantes.add(new Estudiante("victor", "matematica", 1.5));
}

/**
 * Verifica si un estudiante pertenece al curso indicado.
 * Usa Stream con filter y anyMatch para la busqueda.
 */
static boolean perteneceACurso(String nombre, String curso) {
    return estudiantes.stream()
        .filter(e -> e.getNombre().equalsIgnoreCase(nombre) &&
            e.getCurso().equalsIgnoreCase(curso))
        .anyMatch(e -> true);
}

/**
 * Retorna true si el estudiante tiene nota mayor o igual a 4.0.
 */
static boolean estaAprobado(String nombre) {
    return estudiantes.stream()
        .filter(e -> e.getNombre().equalsIgnoreCase(nombre))
        .anyMatch(e -> e.getNota() >= 4.0);
}

/**
 * Retorna true si el estudiante tiene nota menor a 4.0.
 */
static boolean estaReprobado(String nombre) {
    return estudiantes.stream()
        .filter(e -> e.getNombre().equalsIgnoreCase(nombre))
        .anyMatch(e -> e.getNota() < 4.0);
}

/**
 * Lee el nombre de un estudiante por consola, lo busca con Stream
 * y muestra su nota junto con su estado (Aprobado o Reprobado).
 */
static void mostrarEstadoEstudiante() {

```

```

System.out.print("Ingrese el nombre del estudiante: ");
String nombre = sc.nextLine();

// Busca el estudiante con Stream y muestra su informacion si existe
boolean encontrado = estudiantes.stream()
    .filter(e -> e.getNombre().equalsIgnoreCase(nombre))
    .findFirst()
    .map(e -> {
        String estado = e.getNota() >= 4.0 ? "Aprobado" : "Reprobado";
        System.out.println("Nota: " + e.getNota() + " - Estado: " + estado);
        return true;
    })
    .orElse(false);

if (!encontrado) {
    System.out.println("Estudiante no encontrado.");
}
}

/**
 * Solicita dos numeros y una operacion basica, luego muestra el resultado.
 */
static void calculadora() {
    System.out.print("Ingrese el primer numero: ");
    double a = Double.parseDouble(sc.nextLine());
    System.out.print("Ingrese el segundo numero: ");
    double b = Double.parseDouble(sc.nextLine());
    System.out.print("Ingrese la operacion (+, -, *, /): ");
    String op = sc.nextLine().trim();

    switch (op) {
        case "+": System.out.println("Resultado: " + (a + b)); break;
        case "-": System.out.println("Resultado: " + (a - b)); break;
        case "*": System.out.println("Resultado: " + (a * b)); break;
        case "/":
            if (b != 0)
                System.out.println("Resultado: " + (a / b));
            else
                System.out.println("Error: division por cero.");
            break;
        default:
            System.out.println("Operacion no reconocida.");
    }
}

/**
 * Muestra el menu principal y repite hasta que el usuario elija salir.
 */
static void menu() {
    int opcion;
    do {
        System.out.println("\n=== Centro de Apoyo Academico ===");
        System.out.println("1. Consultar estado de un estudiante");
        System.out.println("2. Verificar si pertenece a un curso");
        System.out.println("3. Calculadora");
    }
}

```

```

System.out.println("4. Salir");
System.out.print("Seleccione una opcion: ");
opcion = Integer.parseInt(sc.nextLine());

switch (opcion) {
    case 1:
        mostrarEstadoEstudiante();
        break;
    case 2:
        System.out.print("Nombre del estudiante: ");
        String nombre = sc.nextLine();
        System.out.print("Nombre del curso: ");
        String curso = sc.nextLine();
        if (perteneceACurso(nombre, curso))
            System.out.println(nombre + " pertenece al curso " + curso + ".");
        else
            System.out.println(nombre + " no pertenece al curso " + curso + ".");
        break;
    case 3:
        calculadora();
        break;
    case 4:
        System.out.println("Hasta luego.");
        break;
    default:
        System.out.println("Opcion invalida.");
}
} while (opcion != 4);
}
}

```

4. Cumplimiento de requerimientos

Realicé este apartado solo para apoyarme en la verificación de los cumplimientos, nada mas que para eso. Aun así, puede ver los cumplimientos de forma mas exacta por si es que lo desea.

Criterio (Opción B)	Peso	Implementación en el código
Base de conocimiento (colección Java)	15%	Lista con exactamente 30 objetos Estudiante, variedad en 5 cursos distintos y notas 1.0–6.8
Métodos con Stream: perteneceACurso(), estaAprobado(), estaReproba	20%	Usan stream().filter().anyMatch() en todos los casos

Consulta interactiva (Scanner + Stream)	25%	mostrarEstadoEstudiante() lee con Scanner, busca con findFirst() y muestra nota + estado
Calculadora (+, -, *, /)	15%	calculadora() cubre las 4 operaciones y maneja división por cero
Menú con do-while	10%	menu() usa do-while, repite hasta que el usuario elige la opción 4
Limpieza y estructura (Javadoc)	15%	Cada método tiene comentario Javadoc, código ordenado y sin redundancias

5. Ejecuciones en consola

A continuación, se muestran 5 ejecuciones distintas del programa, cubriendo todos los criterios de la rúbrica.

Ejecución 1 – Consulta Calculadora

Se ingresa la opción número 3 (calculadora). El sistema pide ingresar dos números (23 y 40) en orden consecutivo para luego realizar la operatoria de “suma” ingresando “+” dando como resultado el sistema de: “63.0”.

Criterio demostrado: Consulta interactiva (Scanner + Stream)

```
=== Centro de Apoyo Academico ===
1. Consultar estado de un estudiante
2. Verificar si pertenece a un curso
3. Calculadora
4. Salir
Seleccione una opcion: 3
Ingrese el primer numero: 23
Ingrese el segundo numero: 40
Ingrese la operacion (+, -, *, /): +
Resultado: 63.0
```

Ejecución 2 – Consultar Estado de un estudiante (aprobado)

Se ingresa la opción número 1 (Consultar estado de un estudiante). Se ingresa el nombre “Ana” (nota 6.5). El sistema muestra la nota y el estado “aprobado”.

Criterio demostrado: Consulta interactiva (Scanner + Stream)


```
=== Centro de Apoyo Academico ===
1. Consultar estado de un estudiante
2. Verificar si pertenece a un curso
3. Calculadora
4. Salir
Seleccione una opcion: 1
Ingrese el nombre del estudiante: Ana
Nota: 6.5 - Estado: Aprobado
```

Ejecución 3 – Verificar pertenencia correcta

Se ingresa la opción número 2 (Pertinencia). Se ingresa “nicolas” y el curso “python”. El sistema confirma que pertenece al curso.

Criterio demostrado: Métodos con Stream (perteneceACurso)

```
=== Centro de Apoyo Academico ===
1. Consultar estado de un estudiante
2. Verificar si pertenece a un curso
3. Calculadora
4. Salir
Seleccione una opcion: 2
Nombre del estudiante: nicolas
Nombre del curso: python
nicolas pertenece al curso python.
```

Ejecución 4 – Consultar Estado de un estudiante (aprobado)

Se ingresa la opción número 1 (Consultar estado de un estudiante). Se ingresa el nombre "luis" (nota 3.8). El sistema muestra la nota y el estado "Reprobado".

Criterio demostrado: Métodos con Stream (perteneceACurso)

```
=== Centro de Apoyo Academico ===  
1. Consultar estado de un estudiante  
2. Verificar si pertenece a un curso  
3. Calculadora  
4. Salir  
Seleccione una opcion: 1  
Ingrese el nombre del estudiante: luis  
Nota: 3.8 - Estado: Reprobado
```

Ejecución 5 – Calculadora básica

Se ingresa la opción número 4 (Salir). El sistema finaliza la consulta con un despido de "Hasta luego" hacia el usuario.

Criterio demostrado: Finalización de consulta.

```
=== Centro de Apoyo Academico ===  
1. Consultar estado de un estudiante  
2. Verificar si pertenece a un curso  
3. Calculadora  
4. Salir  
Seleccione una opcion: 4  
Hasta luego.
```